

Capítulo 05

Transformer

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 05 — Transformer

Objetivo de aprendizagem: montar um **Transformer completo**, no estilo do **GPT-2**. Vamos juntar a atenção do Capítulo 4 com o feedforward, adicionar as três peças que faltam — **multi-head attention**, **conexões residuais** e **layer normalization** — e empilhar tudo em profundidade.

Pré-requisitos: Capítulos 1-4. Em especial a self-attention do Cap. 4 (esta aula é a continuação direta dela).

Arquivos:

- `layernorm.py` — LayerNorm do zero, verificada contra o PyTorch
- `transformer.py` — o Transformer completo, treinado nos nomes
- `exercicios.md` — exercícios

1. De onde partimos

O Capítulo 4 terminou com uma conclusão precisa: a atenção **comunica** (move informação entre posições) mas não **computa** (não processa essa informação de forma não-linear). Juntando atenção + feedforward, a loss caiu de 2,099 para 1,913.

Este capítulo transforma esse par em uma **arquitetura** de verdade. Faltam três peças, e nenhuma delas é decorativa — cada uma resolve um problema concreto que aparece quando tentamos empilhar camadas.

Peça	Problema que resolve
Multi-head attention	uma cabeça só aprende um tipo de relação
Conexões residuais	o gradiente se degrada ao atravessar muitas camadas
Layer normalization	as ativações saem de escala e o treino desestabiliza

2. Multi-head attention: várias relações em paralelo

Uma cabeça de atenção produz **um** conjunto de pesos — ou seja, aprende **um** critério de "onde olhar". Mas numa linguagem há vários critérios úteis ao mesmo tempo: a última consoante, a vogal anterior, o começo da palavra...

A solução é rodar **várias cabeças em paralelo** e concatenar o que cada uma trouxe:

```
class MultiHeadAttention(nn.Module):
    def __init__(self, n_head, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(n_head)])
```

```

self.proj = nn.Linear(head_size * n_head, n_embd)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1) # (B, T, n_embd)
    return self.proj(out)

```

Dois detalhes importantes:

O truque da dimensão. Definimos `head_size = n_embd // n_head`. Com `n_embd = 64` e `n_head = 4`, cada cabeça tem dimensão 16, e a concatenação de 4×16 volta a 64. Ou seja: **quatro cabeças pequenas custam o mesmo que uma cabeça grande** — ganhamos diversidade de graça. Não é um trade-off de custo, é um trade-off de "uma relação rica" contra "várias relações mais simples", e na prática o segundo ganha.

A projeção final (`self.proj`). Depois da concatenação, cada cabeça ocupa a sua fatia do vetor, isolada das outras. A camada linear de projeção **mistura** essas fatias, deixando o modelo combinar o que as diferentes cabeças descobriram. Sem ela, as cabeças ficariam em compartimentos estanques.

3. Conexões residuais: um caminho livre para o gradiente

Esta é a ideia mais simples e mais poderosa do capítulo. Em vez de

```
x = sublayer(x) # a camada SUBSTITUI o que veio antes
```

escrevemos

```
x = x + sublayer(x) # a camada ACRESCENTA algo ao que veio antes
```

A diferença parece cosmética. Não é.

Na ida (forward): a sub-camada não precisa mais produzir a representação inteira — ela só precisa calcular uma **correção** ao que já estava lá. Aprender um ajuste é bem mais fácil que reconstruir tudo.

Na volta (backward): lembre do Capítulo 2 — a derivada de uma soma **distribui** o gradiente igualmente para as duas parcelas. Então o `x +` cria uma **rodovia** pela qual o gradiente flui da loss até as primeiras camadas **sem ser multiplicado por nada**. Sem esse caminho, o gradiente precisa atravessar todas as sub-camadas, sendo multiplicado a cada passo — e depois de muitas multiplicações ele encolhe até virar quase zero (o problema do *vanishing gradient*).

É por isso que residuais são o que **viabiliza a profundidade**. A ideia vem das ResNets (2015) e é o motivo de conseguirmos treinar redes com dezenas ou centenas de camadas.

E isso não é só teoria — nós **medimos**. A solução do exercício E2 ([solucoes/e2_ablacoes.py](#)) treina o mesmo modelo com e sem residuais, em duas profundidades:

Configuração

Loss validação

Penalidade por remover

3 blocos, residual ligado	1,903	—
3 blocos, residual desligado	2,197	+0,294
6 blocos, residual ligado	1,887	—
6 blocos, residual desligado	2,773	+0,886

Leia a coluna da direita: com 3 blocos, remover os residuais custa +0,29 de loss. Com 6 blocos, custa +0,89 — **três vezes mais**. A penalidade **cresce com a profundidade**, exatamente como a teoria do gradiente prevê. E note o detalhe mais revelador: **sem** residuais, ir de 3 para 6 blocos *piora* o modelo (2,197 -> 2,773) — mais profundidade vira um estorvo. **Com** residuais, ir de 3 para 6 melhora (1,903 -> 1,887).

Em uma frase: **sem conexões residuais, profundidade atrapalha; com elas, ajuda**. Foi essa peça de duas linhas que destravou o deep learning profundo.

4. Layer normalization

Ao empilhar camadas, as ativações tendem a sair de escala — crescem ou encolhem progressivamente, e o treino fica instável. A **LayerNorm** conserta isso normalizando cada vetor de ativações: subtrai a média e divide pelo desvio padrão.

Implementada do zero (veja [layernorm.py](#)):

```
class LayerNorm(nn.Module):
    def __init__(self, dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.gamma = nn.Parameter(torch.ones(dim)) # ganho aprendido
        self.beta = nn.Parameter(torch.zeros(dim)) # deslocamento aprendido

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        return self.gamma * (x - mean) / torch.sqrt(var + self.eps) + self.beta
```

Três observações:

- **dim=-1**: normalizamos ao longo das **features**, dentro de cada posição. Cada posição de cada sequência é normalizada com os seus próprios números.
- **gamma e beta** são aprendidos: se a normalização não for boa para alguma feature, a rede pode desfazê-la. Normalizar é uma *sugestão*, não uma imposição.
- **eps** evita divisão por zero quando a variância é minúscula.

Rodando `python layernorm.py`, confirmamos que a nossa implementação bate com a do PyTorch e vemos o efeito:

```

=== nossa LayerNorm vs nn.LayerNorm ===
resultados batem? True
diferença máxima: 4.77e-07

=== efeito da normalizacao ===
ANTES : media = +39.858 | desvio = 14.524
DEPOIS: media = -0.000 | desvio = 1.000

```

Por que "Layer" e não "Batch"?

A **BatchNorm** normaliza usando estatísticas do *batch* — ou seja, o resultado de um exemplo **depende dos outros exemplos** processados junto. Para um modelo de linguagem isso é um problema sério: na hora de gerar texto processamos uma sequência sozinha, e a saída não pode depender de quem estava no batch durante o treino.

A LayerNorm não tem esse problema, e o `layernorm.py` prova:

```

=== independencia do batch ===
processar sozinho == processar no batch? True

```

É por isso que Transformers usam LayerNorm.

5. O bloco: comunicação + computação

Agora juntamos tudo. Um **bloco** do Transformer é:

```

class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ff = FeedForward(n_embd)
        self.ln1 = LayerNorm(n_embd)
        self.ln2 = LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x)) # comunicação entre posições
        x = x + self.ff(self.ln2(x)) # computação dentro de cada posição
        return x

```

Leia o `forward` como duas etapas: primeiro as posições **conversam** (atenção), depois cada uma **pensa** sozinha sobre o que ouviu (feedforward). Cada etapa vem embrulhada em LayerNorm (antes) e residual (a soma).

Pre-norm vs post-norm

Note que a LayerNorm vem **antes** da sub-camada (`sublayer(ln(x))`), e não depois. O artigo original de 2017 fazia o contrário (*post-norm*: `ln(x + sublayer(x))`), mas descobriu-se que o **pre-norm** treina de forma bem mais estável — porque assim o caminho residual fica completamente **limpo**, sem nenhuma normalização no meio. O gradiente atravessa a rede sem obstáculo algum. GPT-2 e

praticamente todos os modelos modernos usam pre-norm, e é o que fazemos aqui.

6. O modelo completo

Empilhar blocos e coroar com uma norma final e a camada de saída:

```
class Transformer(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, n_embd)
        self.pos_emb = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head) for _ in range(n_layer)])
        self.ln_f = LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx):
        B, T = idx.shape
        x = self.token_emb(idx) + self.pos_emb(torch.arange(T))
        x = self.blocks(x)
        x = self.ln_f(x)
        x = x[:, -1, :]
        return self.lm_head(x)
```

Isto é um GPT. A arquitetura acima é, em estrutura, a mesma do GPT-2 — e a do GPT-3, e a da maioria dos LLMs em produção. A diferença entre este arquivo e um modelo de fronteira é **escala** (número de blocos, dimensão, tamanho do contexto, volume de dados) e um punhado de refinamentos de engenharia. Não é outra ideia; é a mesma ideia, muito maior.

Para comparar: nosso modelo tem **153.499 parâmetros**, 3 blocos e 4 cabeças. O GPT-2 "small" tem 124 **milhões**, 12 blocos e 12 cabeças, com `n_embd = 768` e contexto de 1024 tokens. Mesmo desenho.

7. Resultados

Rodando `python transformer.py` (cerca de 6 minutos na CPU):

```
parametros: 153499 (3 blocos, 4 cabecas, n_embd=64)
step 0/15000 | loss 3.5052 | 0s
step 1500/15000 | loss 2.4018 | 34s
...
treino concluido em 355s

loss treino = 1.7910
loss validacao = 1.8114
loss teste = 1.8199
```

Comparando com todos os modelos do curso, na **mesma tarefa e mesma métrica**:

Capítulo	Modelo	Contexto	Parâmetros	Loss validação
01	Bigrama (contagem)	1	729	~2,4

03	MLP	3	11.897	1,967
04	Atenção (1 cabeça)	8	11.363	2,099
04	Atenção + feedforward	8	33.255	1,913
05	Transformer	8	153.499	1,811

O Transformer é o **melhor modelo do curso até aqui**, e a trajetória completa é bonita: saímos de 2,4 (contando pares de letras) para 1,811 — construindo cada peça à mão.

Uma ressalva de honestidade: este modelo tem 153 mil parâmetros contra 33 mil do "atenção + feedforward" do Capítulo 4, então **não é uma comparação de orçamento igual**. Parte do ganho vem simplesmente de ser maior. O que a tabela mostra com segurança é que a arquitetura **escala bem** — colocar mais capacidade nesse desenho converte em ganho real, e é exatamente essa propriedade que permite chegar aos modelos de fronteira. Se você quiser a comparação controlada, o exercício E4 varia profundidade e cabeças mantendo o resto fixo.

Note também que treino (1,791) e validação (1,811) continuam próximos: o modelo cresceu bastante, mas o dataset de 64 mil nomes ainda o mantém honesto — sem *overfitting* relevante.

E os nomes gerados ficaram convincentes:

```
rilme, jandir, ludo, raiude, natta, waldes, valdinia, makyane, nerandila
```

Compare com os do bigrama do Capítulo 1 (*cexzma*, *zktahwelo*, *iczisqctoujkwptedo*) — a diferença é evidente.

8. Resumo do capítulo

- **Multi-head attention:** várias cabeças de dimensão `n_embd // n_head` em paralelo, concatenadas e **projetadas** — várias relações pelo mesmo custo de uma.
- **Conexões residuais** (`x + sublayer(x)`): a sub-camada aprende uma *correção*, e o gradiente ganha um caminho direto até o início da rede. É o que viabiliza profundidade.
- **LayerNorm:** normaliza cada posição ao longo das features, com `gamma/beta` aprendidos. Independente do batch (diferente da BatchNorm) — essencial para geração.
- **Bloco** = comunicação (atenção) + computação (feedforward), cada uma com **pre-norm** e residual. O pre-norm mantém o caminho residual limpo.
- **Transformer** = embeddings + N blocos + norma final + `lm_head`. **Isto é a arquitetura do GPT-2**; o resto é escala.
- Resultado: **1,811** de loss — o melhor do curso, contra 1,967 do MLP e ~2,4 do bigrama.

O que vem no Capítulo 6

Temos a arquitetura. Mas ainda estamos alimentando o modelo com **um caractere por token**, o que é um desperdício: sequências ficam longas e o modelo gasta capacidade aprendendo a montar palavras a partir de letras. Modelos reais usam **tokens de sub-palavra**. No **Capítulo 06 — Tokenization** vamos construir um tokenizador **BPE** (*byte pair encoding*) do zero, o mesmo algoritmo que o GPT usa.

-> Antes de seguir, faça os [exercícios](#).

Exercícios — Capítulo 05 (Transformer)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Aviso de tempo: os exercícios que treinam modelos levam alguns minutos cada na CPU. Para experimentar mais rápido, reduza `max_steps` (ex.: 3000) — as *comparações relativas* continuam válidas, só não compare com os números absolutos da apostila.

E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. No `Block`, por que escrevemos `x = x + self.sa(self.ln1(x))` e não `x = self.sa(self.ln1(x))`? O que a soma acrescenta?
2. Por que `head_size = n_embd // n_head`? O que aconteceria com o custo se cada cabeça tivesse `head_size = n_embd`?
3. Qual a diferença entre o que a **atenção** faz e o que o **feedforward** faz dentro de um bloco?

E2 — Sem as conexões residuais

No `Block`, troque as duas linhas por `x = self.sa(self.ln1(x))` e `x = self.ff(self.ln2(x))` (sem o `x +`).

1. A loss final piora? Quanto?
2. Aumente `n_layer` para 6 **sem** os residuais e depois **com**. A diferença cresce com a profundidade?
3. Explique o papel do residual como "caminho livre" para o gradiente.

Solução de referência: [solucoes/e2_ablacoes.py](#).

E3 — Sem LayerNorm

Remova as LayerNorms do bloco (aplique as sub-camadas direto em `x`).

1. O treino fica instável? A loss oscila mais entre passos?
2. Tente também aumentar a learning rate para `3e-3` sem LayerNorm. O que acontece?
3. Por que normalizar as ativações ajuda a estabilizar o treino?

E4 — Profundidade e número de cabeças

Treine com:

- `n_layer` = 1, 3 e 6 (mantendo `n_head` = 4)

- `n_head` = 1, 4 e 8 (mantendo `n_layer` = 3)

1. Anote parâmetros e loss de validação em cada caso.
2. Mais profundidade sempre ajuda? Onde começa o retorno decrescente?
3. Com o **mesmo** `n_embd`, mudar `n_head` altera o número de parâmetros? Por quê? (Dica: olhe `head_size = n_embd // n_head`.)

E5 — Uma cabeça grande vs várias pequenas

Compare `n_head` = 1 (uma cabeça de 64) com `n_head` = 8 (oito de 8), ambos com `n_embd` = 64 — ou seja, praticamente o mesmo número de parâmetros.

1. Qual vai melhor?
2. Por que várias cabeças pequenas podem ganhar de uma grande? (Pense: cada cabeça pode se especializar numa relação diferente.)
3. Existe um limite? O que acontece com `n_head` = 64 (cabeças de dimensão 1)?

E6 — Dropout (regularização, desafio)

O GPT-2 usa **dropout**: durante o treino, zera aleatoriamente uma fração das ativações, o que dificulta a memorização. Adicione `nn.Dropout(0.1)` na saída da projeção da atenção e no fim do feedforward.

1. Com nosso dataset (64 mil nomes), treino e validação já andam juntos. O dropout ajuda, atrapalha ou é indiferente aqui?
2. Em que situação o dropout seria claramente útil? (Releia a lição do Cap. 3 sobre *overfitting*.)
3. Por que o dropout deve ser desligado na avaliação (`model.eval()`)?

E7 — Conte os parâmetros (desafio)

O código informa **153.499** parâmetros. Deduza esse número somando as partes: embeddings (token e posição), por bloco (atenção: `k,q,v,proj`; feedforward: duas camadas; duas LayerNorms), a LayerNorm final e a `lm_head`.

1. Escreva a fórmula em função de `vocab_size`, `block_size`, `n_embd`, `n_head` e `n_layer`.
2. Confira que ela reproduz 153.499.
3. Qual parte domina o total? Se você dobrar `n_embd`, o número de parâmetros dobra ou quadruplica?